```
import socket
import struct
# Create RAW Socket
sniffer = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_IP
# Local machine IP
host = socket.gethostbyname(socket.gethostname())
sniffer.bind((host, 0))
# Include IP headers
sniffer.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
# Enable promiscuous mode (Windows)
sniffer.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)

print("Packet Sniffer Started...\n")
try:
    while True:
        raw_data, addr = sniffer.recvfrom(65535)
        ip_header = raw_data[0:20]
        iph = struct.unpack("!BBHHHBBH4s4s", ip_header)

        version = iph[0] >> 4
        ihl = iph[0] & 0x0F
        ttl = iph[5]
        protocol = iph[6]
        source = socket.inet_ntoa(iph[8])
        destination = socket.inet_ntoa(iph[9])

        print("----------------------------------------")
        print("Version      :", version)
        print("Header Length:", ihl * 4, "bytes")
        print("TTL          :", ttl)
        print("Protocol     :", protocol)
        print("Source IP    :", source)
        print("Destination  :", destination)
        print("Packet Length:", len(raw_data))

except KeyboardInterrupt:
    # Disable promiscuous mode before exiting
    sniffer.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)
    print("\nStopped")
```

Scanned with OKEN Scanner

Exp No: 04

Date: 30/7/26

Setup and Configure a LAN using a switch and Ethernet cables.

Aim: To implement a Packet sniffer using raw socket in Python that captures live network traffic and analyzes Ethernet, IP, TCP, UDP, and ICMP Packet headers.

Prerequisites:-

1. Linux machine, virtual machine (Ubuntu), or WSL2 environments.
2. Python 3 installed.
3. Root / Administrator privileges to create raw socket.
4. Basic understanding of OSI layers, Ethernet frames, IP Packets, and transport-layer protocols (TCP, UDP, ICMP).

Steps:-

1. Set up a Linux-based environment and verify Python installation.
2. Obtain root privileges required for raw socket operations.
3. Create a raw socket capable of capturing Ethernet-layer traffic.
4. Capture incoming network Packets from the network interface.

Scanned with OKEN Scanner

```
----------------------------------------
Version       : 4
Header Length: 20 bytes
TTL           : 62
Protocol      : 17
Source IP     : 142.251.222.206
Destination   : 172.16.10.48
Packet Length: 150
----------------------------------------
Version       : 4
Header Length: 20 bytes
TTL           : 62
Protocol      : 17
Source IP     : 142.251.222.206
Destination   : 172.16.10.48
Packet Length: 53
----------------------------------------
Version       : 4
Header Length: 20 bytes
TTL           : 62
Protocol      : 17
Source IP     : 142.251.222.206
Destination   : 172.16.10.48
Packet Length: 53
----------------------------------------
Version       : 4
Header Length: 20 bytes
TTL           : 128
Protocol      : 17
Source IP     : 172.16.10.48
Destination   : 142.251.222.206
Packet Length: 63
----------------------------------------
Version       : 4
Header Length: 20 bytes
TTL           : 62
Protocol      : 17
Stopped
```

Scanned with OKEN Scanner

5. Extract and analyze the Ethernet header from each captured frame.

6. Check whether the packet contains an IPV4 payload.

7. Parse the IP header to obtain source IP destination IP, TTL, and protocol information.

8. Based on the protocol field, identify whether the packet is TCP, UDP, or ICMP.

9. Extract relevant transport-layer header details such as ports, sequence numbers, flags, packet type, or packet length.

10. Display the analyzed packet information in a readable format.

Result:-

Successfully captured live network packet using raw sockets and analyzed Ethernet, IP, TCP, UDP, and ICMP header information, demonstrating packet sniffing and network traffic monitoring at different layers of the TCP/IP protocol stack.

Scanned with OKEN Scanner